



Name:	Muhammad Hasnat Fakhar
Class/Section:	BSCSev-F23-C
Reg ID:	232436
Subject:	Parallel And Distributing Computing
Task:	LAB TASK 04

Task 1: Unsynchronized Counter

Code:

```
RaceLab > Program.cs > Program
1  using System;
2  using System.Threading;
3
4  0 references
5  class Program
6  {
7      4 references
8      const int NumThreads = 4;
9      2 references
10     const int IncrementsPerThread = 1_000_000;
11
12     3 references
13     static long counter = 0; // shared state
14
15     1 reference
16     static void Worker()
17     {
18         for (int i = 0; i < IncrementsPerThread; i++)
19         {
20             counter++; // unsynchronized read-modify-write
21         }
22     }
23
24     0 references
25     static void Main()
26     {
27         Thread[] threads = new Thread[NumThreads];
28
29         for (int i = 0; i < NumThreads; i++)
30         {
31             threads[i] = new Thread(Worker);
32             threads[i].Start();
33         }
34
35         for (int i = 0; i < NumThreads; i++)
36         {
37             threads[i].Join();
38         }
39
40         long expected = (long)NumThreads * IncrementsPerThread;
41         Console.WriteLine($"Expected: {expected}");
42         Console.WriteLine($"Actual: {counter}");
43         Console.WriteLine($"Lost updates: {expected - counter}");
44     }
45 }
```

Build succeeded in 4.2s

```
PS C:\Users\Hasnat\Documents\Lab04\RaceLab> 1..10
net run --no-build }
Expected: 4000000
Actual: 4000000
Lost updates: 0
Expected: 4000000
Actual: 1701106
Lost updates: 2298894
Expected: 4000000
Actual: 1098675
Lost updates: 2901325
Expected: 4000000
Actual: 1096392
Lost updates: 2903608
Expected: 4000000
Actual: 3914707
Lost updates: 85293
Expected: 4000000
Actual: 1544571
Lost updates: 2455429
Expected: 4000000
Actual: 1793275
Lost updates: 2206725
Expected: 4000000
Actual: 1134334
Lost updates: 2865666
Expected: 4000000
Actual: 1090099
Lost updates: 2909901
Expected: 4000000
Actual: 1371202
Lost updates: 2628798
```

Run	Expected Value	Actual Value	Lost Updates (Expected – Actual)
1	4,000,000	4,000,000	0
2	4,000,000	1,139,561	2,860,439
3	4,000,000	1,129,649	2,870,351
4	4,000,000	1,165,853	2,834,147
5	4,000,000	1,982,542	2,017,458
6	4,000,000	1,067,905	2,932,095
7	4,000,000	1,070,871	2,929,129

8	4,000,000	1,025,099	2,974,901
9	4,000,000	1,214,579	2,785,421
10	4,000,000	1,160,725	2,839,275

Number of logical cores on your machine (Environment.ProcessorCount): 4

Discussion Questions

1. Did the actual value ever equal the expected value? Were any two runs identical? Why is the result non-deterministic even though the source code never changes?

Yes, in Run 1 the actual value equaled the expected value (4,000,000) and there were 0 lost updates. However, no two runs were identical — every other run produced a different actual value and a different number of lost updates. The result is non-deterministic because the operating system schedules threads unpredictably. The interleaving of the three steps of counter++ (load, add, store) varies from run to run, so different updates are lost each time. The source code is fixed, but the timing of thread execution is not.

2. Using the load / add / store breakdown of counter++ from Section 3, explain how one increment can be lost when two threads interleave.

counter++ is three separate operations:

- **Load** the current value from memory into a register.
- **Add** 1 to the register.
- **Store** the register back to memory.

If two threads interleave, both can load the same value before either store. For example:

- Thread A loads counter (reads 5).
- Thread B loads counter (reads 5).
- Thread A adds 1 and stores 6.
- Thread B adds 1 and stores 6.

The final value is 6 instead of 7. Thread B's store overwrote Thread A's update, so one increment is lost. In our runs, millions of such interleaving occurred, causing millions of lost updates.

3. Suppose IncrementsPerThread were reduced to 100. Would you expect the race to disappear, or merely to become harder to observe? Explain why “the output looked correct” is not evidence that a program is free of races.

The race would **not** disappear; it would merely become harder to observe. With only 100 increments per thread, the threads might not overlap enough to cause a visible lost update. The result might occasionally equal the expected value by luck. A correct-looking output does not prove the absence of a race because the race is a property of the code, not of a particular run. The bug is still present; it just did not manifest that time. This is why tools like race detectors analyze memory access and synchronization, not just final output.

Task 2: Tracing the Race and Interpreting the Report

Code :

```
RaceTrace > C# Program.cs > Program > Worker(int) : void
1  ∨ using System;
2  using System.Threading;
3
4  0 references
5  ∨ class Program
6  {
7      6 references
8      |   const int NumThreads = 4;
9      |   5 references
10     |   const int IncrementsPerThread = 250000;
11
12     |   6 references
13     |   static int counter = 0;
14     |   1 reference
15     |   static readonly object gate = new object();
16     |   3 references
17     |   static bool useLock = false;
18     |   4 references
19     |   static int[][] seenLog = new int[NumThreads][];
20
21     |   1 reference
22     |   static void Worker(int id)
23     |   {
24     |       int[] log = seenLog[id];
25     |       for (int i = 0; i < IncrementsPerThread; i++)
26     |       {
27     |           if (useLock)
28     |           {
29     |               lock (gate)
30     |               {
31     |                   int seen = counter;
32     |                   log[i] = seen;
33     |                   counter = seen + 1;
34     |               }
35     |           }
36     |       }
37     |   }
38 }
```

Class Program

```
37 static void Main(string[] args)
38 {
39     useLock = args.Length > 0 && args[0] == "--lock";
40     int total = NumThreads * IncrementsPerThread;
41
42     Thread[] threads = new Thread[NumThreads];
43     for (int t = 0; t < NumThreads; t++)
44     {
45         seenLog[t] = new int[IncrementsPerThread];
46         int id = t;
47         threads[t] = new Thread(() => Worker(id));
48         threads[t].Start();
49     }
50
51     foreach (Thread th in threads) th.Join();
52
53     int[] readCount = new int[total + 1];
54     for (int t = 0; t < NumThreads; t++)
55         for (int i = 0; i < IncrementsPerThread; i++)
56             readCount[seenLog[t][i]]++;
57
58     int collisions = 0;
59     for (int v = 0; v <= total; v++)
60         if (readCount[v] > 1)
61             collisions += readCount[v] - 1;
62
63     string mode = useLock ? "with lock" : "no synchronization";
64     Console.WriteLine($"Mode: {mode}");
65     Console.WriteLine($"Total increments: {total}");
66     Console.WriteLine($"Final counter: {counter}");
67     Console.WriteLine($"Lost updates: {total - counter}");
68     Console.WriteLine($"Collisions: {collisions}");
69
70     // Your task: print first 5 colliding values and their loads
71     Console.WriteLine("First 5 colliding values and their loads:");
72     int printed = 0;
73     for (int v = 0; v <= total && printed < 5; v++)
74     {
75         if (readCount[v] > 1)
76         {
77             printed++;
```

```

75         if (readCount[v] > 1)
76         {
77             printed++;
78             Console.WriteLine($"Value {v} loaded {readCount[v]} times:");
79             for (int t = 0; t < NumThreads; t++)
80             {
81                 for (int i = 0; i < IncrementsPerThread; i++)
82                 {
83                     if (seenLog[t][i] == v)
84                     {
85                         Console.WriteLine($" Thread {t}, loop index {i}");
86                     }
87                 }
88             }
89         }
90     }
91 }
92 }

```

```

● PS C:\Users\Hasnat\Documents\Lab04\RaceTrace> dotnet run --no-build
Mode: no synchronization
Total increments: 1000000
Final counter: 402011
Lost updates: 597989
Collisions: 597989
First 5 colliding values and their loads:
Value 10218 loaded 2 times:
  Thread 0, loop index 10218
  Thread 1, loop index 0
Value 10219 loaded 2 times:
  Thread 0, loop index 10219
  Thread 1, loop index 1
Value 10220 loaded 2 times:
  Thread 0, loop index 10220
  Thread 1, loop index 2
Value 10221 loaded 2 times:
  Thread 0, loop index 10221
  Thread 1, loop index 3
Value 10222 loaded 2 times:
  Thread 0, loop index 10222
  Thread 1, loop index 4

```

```

● PS C:\Users\Hasnat\Documents\Lab04\RaceTrace> dotnet run --no-build

```

```
PS C:\Users\Hasnat\Documents\Lab04\RaceTrace> dotnet run --no-build
Mode: no synchronization
Total increments: 1000000
Final counter: 394988
Lost updates: 605012
Collisions: 605012
First 5 colliding values and their loads:
Value 3050 loaded 2 times:
  Thread 0, loop index 3050
  Thread 1, loop index 0
Value 3051 loaded 2 times:
  Thread 0, loop index 3051
  Thread 1, loop index 1
Value 3052 loaded 2 times:
  Thread 0, loop index 3052
  Thread 1, loop index 2
Value 3053 loaded 2 times:
  Thread 0, loop index 3053
  Thread 1, loop index 3
Value 3054 loaded 2 times:
  Thread 0, loop index 3054
  Thread 1, loop index 4
```

```
PS C:\Users\Hasnat\Documents\Lab04\RaceTrace> dotnet run --no-build
Mode: no synchronization
```

```
PS C:\Users\Hasnat\Documents\Lab04\RaceTrace> dotnet run --no-build
Mode: no synchronization
Total increments: 1000000
Final counter: 408692
Lost updates: 591308
Collisions: 591308
First 5 colliding values and their loads:
Value 3804 loaded 2 times:
  Thread 0, loop index 3804
  Thread 1, loop index 0
Value 3805 loaded 2 times:
  Thread 0, loop index 3805
  Thread 1, loop index 1
Value 3806 loaded 2 times:
  Thread 0, loop index 3806
  Thread 1, loop index 2
Value 3807 loaded 2 times:
  Thread 0, loop index 3807
  Thread 1, loop index 3
Value 3808 loaded 2 times:
  Thread 0, loop index 3808
  Thread 1, loop index 4
```

Results

Item	Run 1	Run 2	Run 3
Total increments	1,000,000	1,000,000	1,000,000
Final counter value	402,011	394,988	408,692
Lost updates (total – final)	597,989	605,012	591,308
Collisions	597,989	605,012	591,308
First colliding value	10,218	3,050	3,804
Threads that loaded that value	Thread 0 (index 10218) and Thread 1 (index 0)	Thread 0 (index 3050) and Thread 1 (index 0)	

Discussion Questions

1. What does a collision mean in terms of the load / add / store steps? Describe, using one real collision from your output, how two threads ended up loading the same value.

A collision means two increments loaded the same counter value before either stored its result. In Run 1, value 10218 was loaded by Thread 0 at loop index 10218 and by Thread 1 at loop index 0. Both threads performed the load step and read 10218. Then both added 1 and both stored 10219. The second store overwrote the first, so one increment was lost. This is a collision: the same value was loaded more than once.

2. Is the number of lost updates equal to the number of collisions in your results? Compare the two and explain any difference (hint: consider what a thread with a stale value does when it stores).

In all three runs, the number of lost updates equals the number of collisions exactly ($597,989 = 597,989$; $605,012 = 605,012$; $591,308 = 591,308$). In this simple counter, each collision corresponds to exactly one lost update because only one thread overwrites another's result. However, they could differ in general: if a thread with a stale value stores its result after multiple other updates have occurred, it could overwrite more than one increment, causing more lost updates than collisions. Here, equality holds because each collision involves exactly two threads loading the same value and one of them losing its update.

3. Suppose a run of this detector reported zero collisions. Would that prove the program is race-free? Explain, and contrast a tool that only sees races that happened in a run with one (such as ThreadSanitizer for C/C++) that analyzes whether accesses are ordered by synchronization.

No, zero collisions in one run does not prove the program is race-free. It only means that in that particular execution, no two increments loaded the same value. The race condition may still exist in the code; it just did not manifest. A tool like this detector only sees races that actually happened. In contrast, ThreadSanitizer analyzes memory accesses and synchronization ordering, so it can detect potential races even if they do not cause visible errors in a given run. It checks whether accesses are ordered by synchronization, not just whether a collision occurred.

4. Why was it important that each thread wrote to its own private log rather than all threads appending to one shared list?

Writing to a shared list would introduce a new race condition on the log itself and would require synchronization (e.g., a lock) to protect it. That synchronization would change the timing of the original race and could hide or alter the behavior we are trying to observe. By giving each thread its own private log, we avoid introducing new sharing, keep the instrumentation thread-safe without locks, and preserve the original race on counter as much as possible.

Task 3: Fixing the Race with a Lock (Critical Section)

Code :

```
1  using System;
2  using System.Threading;
3
4  class Program
5  {
6      const int NumThreads = 4;
7      const int IncrementsPerThread = 1_000_000;
8
9      static long counter = 0;
10     static readonly object counterLock = new object();
11
12     static void Worker()
13     {
14         for (int i = 0; i < IncrementsPerThread; i++)
15         {
16             lock (counterLock)
17             {
18                 counter++;
19             }
20         }
21     }
22
23     static void Main()
24     {
25         Thread[] threads = new Thread[NumThreads];
26
27         for (int i = 0; i < NumThreads; i++)
28         {
29             threads[i] = new Thread(Worker);
30             threads[i].Start();
31         }
32     }
33 }
```


OUTPUT 2:

```
● PS C:\Users\Hasnat\Documents\Lab04\MutexFix> dotnet run -- --lock
Expected: 4000000
Actual: 4000000
Lost updates: 0
● PS C:\Users\Hasnat\Documents\Lab04\MutexFix> dotnet run -- --lock
Expected: 4000000
Actual: 4000000
Lost updates: 0
● PS C:\Users\Hasnat\Documents\Lab04\MutexFix> dotnet run -- --lock
Expected: 4000000
Actual: 4000000
Lost updates: 0
❖ PS C:\Users\Hasnat\Documents\Lab04\MutexFix> |
```

Results

MutexFix Results

Run	Expected Value	Actual Value	Correct? (Y/N)
1	4,000,000	4,000,000	Y
2	4,000,000	4,000,000	Y
3	4,000,000	4,000,000	Y
4	4,000,000	4,000,000	Y
5	4,000,000	4,000,000	Y
6	4,000,000	4,000,000	Y
7	4,000,000	4,000,000	Y
8	4,000,000	4,000,000	Y
9	4,000,000	4,000,000	Y
10	4,000,000	4,000,000	Y

RaceTrace --lock Results

RaceTrace --lock	Total Increments	Final Counter	Collisions
Run 1	1,000,000	1,000,000	0
Run 2	1,000,000	1,000,000	0
Run 3	1,000,000	1,000,000	0

Discussion Questions

1. Was the actual value equal to the expected value in all 10 runs? Why is this on its own not conclusive proof that the race is fixed, and what additional evidence did the --lock trace provide? Is even that evidence a proof?

Yes, in all 10 runs the actual value equaled the expected value (4,000,000) and lost updates were 0. However, correct results alone are not conclusive proof that the race is fixed. A racy program can occasionally produce correct results by luck, especially if the timing happens to avoid interleaving. The --lock trace provides stronger evidence: it shows zero collisions and a final counter equal to the total. This means that no two increments loaded the same value, which is consistent with proper mutual exclusion. Even this is not a formal proof, because it only covers the runs that were executed; there could still be a latent race that did not manifest. But it is much stronger evidence than the final result alone.

2. Main() reads counter after the joins without taking the lock. Why is that safe here? (Hint: what does Thread.Join() guarantee about the finished threads' writes?)

Thread.Join() blocks the calling thread until the target thread terminates. It also guarantees that all writes made by the finished thread are visible to the thread that called Join(). So after all worker threads have been joined, the main thread sees the final value of counter correctly, without needing to take the lock. The lock is only needed to protect concurrent access during execution; after all threads have stopped, there is no concurrency.

3. What would happen to correctness if only three of the four threads used the lock and one still executed a bare counter++? What if each thread locked a different object?

If only three threads used the lock and one did a bare counter++, the race would still exist because the unlocked thread could interleave its load/add/store with the locked threads, causing lost updates. Mutual exclusion requires all threads that access the shared state to use the same lock. If each thread locked a different object, there would be no mutual exclusion at all, because each thread would acquire its own separate lock, allowing them to enter the critical section simultaneously. The race would remain completely unfixed.

4. The lock is placed around each single increment rather than around the whole loop. What is the trade-off between the two placements in terms of correctness, parallelism, and lock overhead?

- **Correctness:** Both placements are correct as long as all threads use the same lock. Locking around each increment ensures that each increment is atomic. Locking around the whole loop would make the entire loop atomic, which is also correct but unnecessarily restrictive.
- **Parallelism:** Locking around each increment allows threads to interleave between increments, which can improve parallelism if [there](#) is other work to do. Locking around the whole loop serializes the entire loop, so only one thread can run its loop at a time, reducing parallelism.
- **Lock overhead:** Locking around each increment incurs the overhead of acquiring and releasing the lock 1,000,000 times per thread. Locking around the whole loop incurs this overhead only once per thread, which is much cheaper. However, the whole-loop lock reduces concurrency, which may be worse for overall performance if there are multiple cores. The best choice depends on the workload. For a simple counter, the per-increment lock is correct but has high overhead; the atomic version in Task 4 is often faster.

Task 4: Atomic Increment vs. Critical Section

Code :

```
CounterBench > Program.cs > Program > IncrementsPerThread : int
1  using System;
2  using System.Diagnostics;
3  using System.Threading;
4
5  0 references
6  class Program
7  {
8      4 references
9      const int NumThreads = 4;
10     3 references
11     const int IncrementsPerThread = 1000000;
12
13     5 references
14     static long counter = 0;
15     1 reference
16     static readonly object counterLock = new object();
17
18     2 references
19     static void LockWorker()
20     {
21         for (int i = 0; i < IncrementsPerThread; i++)
22         {
23             lock (counterLock)
24             {
25                 counter++;
26             }
27         }
28     }
29
30     2 references
31     static void InterlockedWorker()
32     {
33         for (int i = 0; i < IncrementsPerThread; i++)
34         {
35             Interlocked.Increment(ref counter);
36         }
37     }
38
39     4 references
40     static double RunVersion(string name, ThreadStart worker)
41     {
42         counter = 0;
43         Thread[] threads = new Thread[NumThreads];
```

```

36
37     Stopwatch sw = Stopwatch.StartNew();
38
39     for (int i = 0; i < NumThreads; i++)
40     {
41         threads[i] = new Thread(worker);
42         threads[i].Start();
43     }
44
45     for (int i = 0; i < NumThreads; i++)
46     {
47         threads[i].Join();
48     }
49
50     sw.Stop();
51
52     long expected = (long)NumThreads * IncrementsPerThread;
53     double ms = sw.Elapsed.TotalMilliseconds;
54     Console.WriteLine($"{name}: expected={expected} actual={counter} match={expected == counter} time={ms:F2} ms");
55     return ms;
56 }
57
58 0 references
59 static void Main()
60 {
61     // Warm-up pass (JIT compilation)
62     RunVersion("warm-up lock", LockWorker);
63     RunVersion("warm-up Interlocked", InterlockedWorker);
64
65     RunVersion("lock", LockWorker);
66     RunVersion("Interlocked.Increment", InterlockedWorker);

```

PS C:\Users\Hasnat\Documents\Lab04>

• PS C:\Users\Hasnat\Documents\Lab04> cd CounterBench

• PS C:\Users\Hasnat\Documents\Lab04\CounterBench> dotnet build -c Release

>> 1..5 | ForEach-Object { dotnet run --no-build -c Release }

Restore complete (0.7s)

CounterBench net10.0 succeeded (4.4s) → bin\Release\net10.0\CounterBench.dll

Build succeeded in 6.4s

warm-up lock: expected=4000000 actual=4000000 match=True time=193.44 ms

warm-up Interlocked: expected=4000000 actual=4000000 match=True time=83.38 ms

lock: expected=4000000 actual=4000000 match=True time=169.71 ms

Interlocked.Increment: expected=4000000 actual=4000000 match=True time=72.17 ms

warm-up lock: expected=4000000 actual=4000000 match=True time=162.50 ms

warm-up Interlocked: expected=4000000 actual=4000000 match=True time=76.87 ms

lock: expected=4000000 actual=4000000 match=True time=158.69 ms

Interlocked.Increment: expected=4000000 actual=4000000 match=True time=79.41 ms

warm-up lock: expected=4000000 actual=4000000 match=True time=159.93 ms

warm-up Interlocked: expected=4000000 actual=4000000 match=True time=82.18 ms

lock: expected=4000000 actual=4000000 match=True time=157.38 ms

Interlocked.Increment: expected=4000000 actual=4000000 match=True time=76.60 ms

warm-up lock: expected=4000000 actual=4000000 match=True time=158.50 ms

warm-up Interlocked: expected=4000000 actual=4000000 match=True time=81.56 ms

lock: expected=4000000 actual=4000000 match=True time=158.21 ms

Interlocked.Increment: expected=4000000 actual=4000000 match=True time=79.87 ms

warm-up lock: expected=4000000 actual=4000000 match=True time=141.87 ms

warm-up Interlocked: expected=4000000 actual=4000000 match=True time=82.78 ms

lock: expected=4000000 actual=4000000 match=True time=137.02 ms

Interlocked.Increment: expected=4000000 actual=4000000 match=True time=79.56 ms

PS C:\Users\Hasnat\Documents\Lab04\CounterBench> dotnet build -c Release

• >> 1..5 | ForEach-Object { dotnet run --no-build -c Release }

Restore complete (0.7s)

CounterBench net10.0 succeeded (0.5s) → bin\Release\net10.0\CounterBench.dll

Results

Run	lock Time (ms)	Interlocked.Increment Time (ms)
1	143.93	74.49
2	149.97	65.31
3	142.59	68.16
4	156.95	78.90
5	158.38	73.81
Average	150.36	72.13
Correct in all runs? (Y/N)	Y	Y

Ratio (average lock time / average Interlocked time): $150.36 / 72.13 \approx 2.08$

Number of threads and increments per thread used: 4 threads, 1,000,000 increments per thread

Discussion Questions

1. Did both versions produce the correct total in every run? Compare this with the unsynchronized program in Task 1.

Yes, both the lock version and the Interlocked.Increment version produced the correct total (4,000,000) in every run, as shown by match=True for all runs. This is in sharp contrast to the unsynchronized program in Task 1, which almost never reached the expected total (only 1 out of 10 runs was correct) and produced different lost update counts each run. The lock and the atomic increment both ensure that no updates are lost.

2. Which version was faster on your machine, and by roughly what factor? Give two reasons why an atomic increment tends to be cheaper than a lock/unlock pair around the same increment.

The Interlocked.Increment version was faster. On average, the lock version took 150.36 ms while the Interlocked version took 72.13 ms, so the lock version was about **2.08× slower** (or the Interlocked version was about 2× faster). Two reasons:

- **Hardware support:** Interlocked.Increment compiles to a single atomic hardware instruction (e.g., LOCK XADD on x86), which is very fast. A lock requires acquiring and releasing a monitor, which involves more overhead, including potential kernel transitions and context switches if contended.
- **No blocking:** The atomic increment never blocks the thread; it retries in hardware if needed. A lock can cause threads to block and be rescheduled, adding latency and overhead.

3. Even the Interlocked version is far slower than a single thread incrementing an unshared variable. What causes this, given that all threads write the same memory location? How does the one-slot-per-thread approach from Lab 03, Task 2 avoid it?

The slowdown is caused by **cache coherence traffic and contention**. When multiple threads increment the same memory location, the cache line containing those variable bounces between cores. Each atomic increment must invalidate the copy in other cores' caches and fetch the latest value, which is expensive. This serializes the

updates at the hardware level. The one-slot-per-thread approach from Lab 03, Task 2 gives each thread its own private variable (e.g., an array element) to update. Since each thread writes to a different memory location, there is no contention on the same cache line, and the updates can proceed in parallel without cache coherence overhead. At the end, the per-thread results are combined.

4. An atomic increment protects one variable. Give an example of an update that Interlocked cannot make safe, but a lock can (for instance, an update that must change two related fields together) and explain why.

Example: transferring money between two bank accounts. You need to decrement one account's balance and increment another's atomically. Interlocked can only operate on a single variable at a time. If you do Interlocked.Decrement(ref accountA) and then Interlocked.Increment(ref accountB), another thread could observe the intermediate state where money has left account A but not yet arrived in account B, violating consistency. A lock can protect the entire critical section that reads and writes both balances, ensuring the two updates appear as one atomic operation.

5. How would you expect the ratio of lock time to Interlocked time to change with 1 thread, and with more threads than cores? Justify your answer, and if time permits, test it by changing NumThreads to 1 and to 8.

- **With 1 thread:** There is no contention, so the lock is uncontended and very cheap. The ratio of lock time to Interlocked time should be close to 1, because both are just incrementing without competition. The lock overhead is minimal when uncontended.
- **With more threads than cores:** Contention increases. The lock will cause more threads to block and be rescheduled, adding significant overhead. The Interlocked version will also suffer from cache-line contention, but it may still be cheaper because it does not cause blocking. The ratio (lock time / Interlocked time) should increase, meaning the lock becomes relatively slower compared to the atomic version. However, if contention is extremely high, the atomic version may also degrade due to cache-line bouncing, but it typically remains faster than a lock under high contention.

Repo Link :

<https://github.com/mhasnat013/Muhammad-Hasnat-Fakhar.git>